

5

REDUTIBILIDADE

No Capítulo 4, estabelecemos a máquina de Turing como nosso modelo de um computador de propósito geral. Apresentamos diversos exemplos de problemas que são solúveis por máquinas de Turing e demos um exemplo de um problema, A_{MT} , que é computacionalmente insolúvel. Neste capítulo examinaremos vários problemas insolúveis adicionais. Ao fazer isso, introduzimos o método principal de provar que problemas são computacionalmente insolúveis. Ele é chamado *redutibilidade*.

Uma *redução* é uma maneira de converter um problema em outro de forma que uma solução para o segundo problema possa ser usada para resolver o primeiro. Essas redutibilidades aparecem frequentemente no dia-a-dia, mesmo que em geral não nos refiramos a elas dessa forma.

Por exemplo, suponha que você deseje se orientar em uma nova cidade. Você sabe que seria fácil fazer isso se tivesse um mapa. Conseqüentemente, você pode reduzir o problema de se orientar na cidade ao problema de se obter um mapa da cidade.

A redutibilidade sempre envolve dois problemas, que denominamos A e B . Se A se reduz a B , podemos usar uma solução para B para resolver A . Assim, em nosso exemplo, A é o problema de se orientar na cidade e B é o problema de se obter um mapa. Note que redutibilidade não diz nada sobre resolver A ou B sozinhos, mas somente sobre a solubilidade de A na presença de uma solução para B .

O que se segue são exemplos adicionais de redutibilidades. O problema de se viajar de Boston a Paris se reduz ao problema de se comprar uma passagem

aérea entre as duas cidades. Esse problema, por sua vez, se reduz ao problema de se ganhar o dinheiro para a passagem. E esse último problema se reduz ao problema de se encontrar um emprego.

A redutibilidade também ocorre em problemas matemáticos. Por exemplo, o problema de se medir a área de um retângulo se reduz ao problema de se medir seu comprimento e largura. O problema de se resolver um sistema de equações lineares se reduz ao problema de se inverter uma matriz.

A redutibilidade desempenha um importante papel na classificação de problemas por decidibilidade e, posteriormente, em teoria da complexidade também. Quando A é redutível a B , resolver A não pode ser mais difícil que resolver B , porque uma solução para B dá uma solução para A . Em termos de teoria da computabilidade, se A for redutível a B e B for decidível, A também será decidível. Equivalentemente, se A for indecidível e redutível a B , B será indecidível. Essa última versão é chave para se provar que vários problemas são indecidíveis.

Em resumo, nosso método para provar que um problema é indecidível será mostrar que algum outro problema já conhecido como sendo indecidível se reduz a ele.

5.1

PROBLEMAS INDECIDÍVEIS DA TEORIA DE LINGUAGENS

Já estabelecemos a indecidibilidade de A_{MT} , o problema de se determinar se uma máquina de Turing aceita uma dada entrada. Vamos considerar um problema relacionado, $PARA_{MT}$, o problema de se determinar se uma máquina de Turing pára (aceitando ou rejeitando) sobre uma dada entrada.¹ Usamos a indecidibilidade de A_{MT} para provar a indecidibilidade de $PARA_{MT}$ reduzindo A_{MT} a $PARA_{MT}$. Seja

$$PARA_{MT} = \{ \langle M, w \rangle \mid M \text{ é uma MT e } M \text{ pára sobre a entrada } w \}.$$

TEOREMA 5.1

$PARA_{MT}$ é indecidível.

¹ Na Seção 4.2, usamos o termo *problema da parada* para a linguagem A_{MT} , muito embora $PARA_{MT}$ seja o real problema da parada. Daqui em diante, distinguiremos entre os dois chamando A_{MT} de *problema da aceitação*.

IDÉIA DA PROVA Essa prova é por contradição. Supomos que $PARA_{MT}$ seja decidível e usamos essa suposição para mostrar que A_{MT} é decidível, contradizendo o Teorema 4.11. A idéia-chave é mostrar que A_{MT} é redutível a $PARA_{MT}$.

Vamos supor que temos uma MT R que decide $PARA_{MT}$. Então usamos R para construir S , uma MT que decide A_{MT} . Para ter uma idéia da forma de construir S , faça de conta que você seja S . Sua tarefa é decidir A_{MT} . Você recebe uma entrada da forma $\langle M, w \rangle$. Você tem de dar como resposta *aceite* se M aceita w e deve responder *rejeite* se M entra em *loop* ou rejeita sobre w . Tente simular M sobre w . Se ela aceita ou rejeita, faça o mesmo. Mas você pode não ser capaz de determinar se M está em *loop* e, nesse caso, sua simulação não terminará. Isso é ruim, pois você é um decisor e, portanto, nunca lhe é permitido entrar em *loop*. Assim, essa idéia, por si só, não funciona.

Em vez disso, use a suposição de que você tem a MT R que decide $PARA_{MT}$. Com R , você pode testar se M pára sobre w . Se R indica que M não pára sobre w , rejeite, porque $\langle M, w \rangle$ não está em A_{MT} . Entretanto, se R indica que M pára sobre w , você pode fazer a simulação sem qualquer perigo de entrar em *loop*.

Conseqüentemente, se a MT R existe, podemos decidir A_{MT} ; mas sabemos que A_{MT} é indecidível. Em virtude dessa contradição, podemos concluir que R não existe. Logo, $PARA_{MT}$ é indecidível.

PROVA Vamos supor, para os propósitos de obter uma contradição, que a MT R decida $PARA_{MT}$. Construímos a MT S para decidir A_{MT} , com S operando da seguinte forma.

S = “Sobre a entrada $\langle M, w \rangle$, uma codificação de uma MT M e uma cadeia w :

1. Rode MT R sobre a entrada $\langle M, w \rangle$.
2. Se R rejeita, *rejeite*.
3. Se R aceita, simule M sobre w até que ela pare.
4. Se M aceitou, *aceite*; se M rejeitou, *rejeite*.”

Claramente, se R decide $PARA_{MT}$, então S decide A_{MT} . Como A_{MT} é indecidível, $PARA_{MT}$ também deve ser indecidível.

O Teorema 5.1 ilustra nossa estratégia para provar que um problema é indecidível. Essa estratégia é comum à maioria das provas de indecidibilidade, exceto no caso da indecidibilidade da própria A_{MT} , que é provada diretamente através do método da diagonalização.

Agora, apresentamos vários outros teoremas e suas provas como exemplos adicionais do método da redutibilidade para provar indecidibilidade. Seja

$$V_{MT} = \{ \langle M \rangle \mid M \text{ é uma MT e } L(M) = \emptyset \}.$$

TEOREMA 5.2

V_{MT} é indecidível.

IDÉIA DA PROVA Seguimos o padrão adotado no Teorema 5.1. Supomos, para o propósito de obter uma contradição, que V_{MT} é decidível e, aí então, mostramos que A_{MT} é decidível — uma contradição. Seja R uma MT que decide V_{MT} . Usamos R para construir a MT S que decide A_{MT} . Como S funcionará quando ela receber a entrada $\langle M, w \rangle$?

Uma idéia é S rodar R sobre a entrada $\langle M \rangle$ e ver se ela aceita. Se aceita, sabemos que $L(M)$ é vazia e, por conseguinte, que M não aceita w . Mas, se R rejeita $\langle M \rangle$, tudo o que sabemos é que $L(M)$ não é vazia e, conseqüentemente, que M aceita alguma cadeia, porém, ainda não sabemos se M aceita a cadeia específica w . Dessa forma, precisamos usar uma idéia diferente.

Em vez de rodar R sobre $\langle M \rangle$, rodamos R sobre uma modificação de $\langle M \rangle$. Modificamos $\langle M \rangle$ para garantir que M rejeite todas as cadeias, exceto w , mas que sobre a entrada w ela funcione normalmente. Então usamos R para determinar se a máquina modificada reconhece a linguagem vazia. A única cadeia que a máquina agora aceita é w , e, portanto, sua linguagem será não vazia sse ela aceita w . Se R aceita quando é alimentada com uma descrição da máquina modificada, sabemos que a máquina modificada não aceita nada e que M não aceita w .

PROVA Vamos escrever a máquina modificada descrita na idéia da prova utilizando nossa notação-padrão. Chamemo-la M_1 .

M_1 = “Sobre a entrada x :

1. Se $x \neq w$, rejeite.
2. Se $x = w$, rode M sobre a entrada w e aceite se M aceita.”

Essa máquina tem a cadeia w como parte de sua descrição. Ela conduz o teste $x = w$ da maneira óbvia, fazendo uma varredura na entrada e comparando-a caractere por caractere com w para determinar se elas são iguais.

Juntando tudo, supomos que a MT R decide V_{MT} e construímos a MT S que decide A_{MT} da seguinte forma.

S = “Sobre a entrada $\langle M, w \rangle$, uma codificação de uma MT M e uma cadeia w :

1. Use a descrição de M e w para construir a MT M_1 descrita anteriormente.
2. Rode R sobre a entrada $\langle M_1 \rangle$.
3. Se R aceita, rejeite; se R rejeita, aceite.”

Note que S , na realidade, tem de ser capaz de computar uma descrição de M_1 a partir de uma descrição de M e w . Ela é capaz de fazê-lo, porque só precisa adicionar novos estados a M que realizem o teste $x = w$.

Se R fosse um decisor para V_{MT} , S seria um decisor para A_{MT} . Um decisor para A_{MT} não pode existir, portanto sabemos que V_{MT} deve ser indecidível.

Outro problema computacional interessante concernente a máquinas de Turing diz respeito a determinar se uma dada máquina de Turing reconhece uma linguagem que também pode ser reconhecida por um modelo computacional mais simples. Por exemplo, seja $REGULAR_{MT}$ o problema de se determinar se uma dada máquina de Turing tem um autômato finito equivalente. Esse problema é o mesmo que determinar se a máquina de Turing reconhece uma linguagem regular. Seja

$$REGULAR_{MT} = \{ \langle M \rangle \mid M \text{ é uma MT e } L(M) \text{ é uma linguagem regular} \}.$$

TEOREMA 5.3

$REGULAR_{MT}$ é indecidível.

IDÉIA DA PROVA Como de costume para teoremas de indecidibilidade, esta prova é por redução a partir de A_{MT} . Supomos que $REGULAR_{MT}$ seja decidível por uma MT R e usamos essa suposição para construir uma MT S que decide A_{MT} . Menos óbvio agora é como usar a capacidade de R para auxiliar S na sua tarefa. Não obstante, podemos fazê-lo.

A idéia é S tomar sua entrada $\langle M, w \rangle$ e modificar M de modo que a MT resultante reconheça uma linguagem regular se e somente se M aceita w . Chamamos a máquina modificada M_2 . Projetamos M_2 para reconhecer a linguagem não-regular $\{0^n 1^n \mid n \geq 0\}$ se M não aceita w , e para reconhecer a linguagem regular Σ^* se M aceita w . Temos de especificar como S pode construir tal M_2 a partir de M e w . Aqui, M_2 funciona aceitando automaticamente todas as cadeias em $\{0^n 1^n \mid n \geq 0\}$. Adicionalmente, se M aceita w , M_2 aceita todas as outras cadeias.

PROVA Supomos que R seja uma MT que decide $REGULAR_{MT}$ e construimos a MT S para decidir A_{MT} . Então, S funciona da seguinte maneira.

$S =$ "Sobre a entrada $\langle M, w \rangle$, onde M é uma MT e w é uma cadeia:

1. Construa a seguinte MT M_2 .

$M_2 =$ "Sobre a entrada x :

1. Se x tem a forma $0^n 1^n$, aceite.
2. Se x não tem essa forma, rode M sobre a entrada w e aceite se M aceita w ."

2. Rode R sobre a entrada $\langle M_2 \rangle$.

3. Se R aceita, aceite; se R rejeita, rejeite."

Similarmente, os problemas de se testar se a linguagem de uma máquina é uma linguagem livre-do-contexto, uma linguagem decidível ou mesmo uma linguagem finita podem ser mostrados indecidíveis com provas similares. Na verdade, um resultado geral, chamado teorema de Rice, afirma que testar *qualquer propriedade* das linguagens reconhecidas por máquinas de Turing é indecidível. Daremos o teorema de Rice no Problema 5.28.

Até o momento, nossa estratégia para provar a indecibilidade de linguagens envolve uma redução a partir de A_{MT} . Às vezes, reduzir a partir de alguma outra linguagem indecidível, tal como V_{MT} , é mais conveniente quando estamos mostrando que certas linguagens são indecidíveis. O teorema a seguir mostra que testar a equivalência de duas máquinas de Turing é um problema indecidível. Poderíamos prová-lo por uma redução a partir de A_{MT} , mas usamos essa oportunidade para dar um exemplo de uma prova de indecibilidade por redução a partir de V_{MT} . Seja

$$EQ_{MT} = \{ \langle M_1, M_2 \rangle \mid M_1 \text{ e } M_2 \text{ são MTs e } L(M_1) = L(M_2) \}.$$

TEOREMA 5.4

EQ_{MT} é indecidível.

IDÉIA DA PROVA Mostre que, se EQ_{MT} fosse decidível, V_{MT} também seria, dando uma redução de V_{MT} para EQ_{MT} . A idéia é simples. V_{MT} é o problema de se determinar se a linguagem de uma MT é vazia. EQ_{MT} é o problema de se determinar se as linguagens de duas MTs são iguais. Se uma dessas linguagens acontece de ser $\emptyset$, vamos dar no problema de se determinar se a linguagem da outra máquina é vazia — ou seja, o problema V_{MT} . Dessa forma, em um certo sentido, o problema V_{MT} é um caso especial do problema EQ_{MT} no qual uma das máquinas é fixada para reconhecer a linguagem vazia. Essa idéia faz com que a redução seja fácil de ser feita.

PROVA Supomos que a MT R decide EQ_{MT} e construímos a MT S para decidir V_{MT} da seguinte forma.

$S =$ “Sobre a entrada $\langle M \rangle$, onde M é uma MT:

1. Rode R sobre a entrada $\langle M, M_1 \rangle$, onde M_1 é uma MT que rejeita todas as entradas.
2. Se R aceita, *aceite*; se R rejeita, *rejeite*.”

Se R decide EQ_{MT} , S decide V_{MT} . Mas V_{MT} é indecidível pelo Teorema 5.2, portanto EQ_{MT} também tem de ser indecidível.